

Avail List Operations in C

```
#include <stdio.h>

#include <stdlib.h>

#define SIZE 10

struct Node
{
    int data;
    int next;
};

struct Node list[SIZE];

int start = -1;

int avail = 0;

void initialize()
{
    for (int i = 0; i < SIZE - 1; i++)
    {
        list[i].next = i + 1;
    }
    list[SIZE - 1].next = -1;
}
```

```
int getNode()
{
    if (avail == -1)
    {
        printf("No free nodes available!\n");
        return -1;
    }

    int newNode = avail;
    avail = list[avail].next;
    return newNode;
}
```

```
void freeNode(int index)
{
    list[index].next = avail;
    avail = index;
}
```

```
void insertAtBeginning(int value)
{
    int newNode = getNode();
    if (newNode == -1)
        return;

    list[newNode].data = value;
```

```
list[newNode].next = start;

start = newNode;

printf("Inserted %d at the beginning.\n", value);
}
```

```
void traverse()
{
    if (start == -1)
    {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");

    int temp = start;

    while (temp != -1)
    {
        printf("%d ", list[temp].data);

        temp = list[temp].next;
    }

    printf("\n");
}
```

```
void deleteFirst()
{
    if (start == -1)
```

```
{
    printf("List is empty. Cannot delete.\n");
    return;
}

int temp = start;
start = list[start].next;
freeNode(temp);
printf("Deleted first element successfully.\n");
}
```

```
void displayAvailable()
{
    printf("Available List: ");
    int temp = avail;
    while (temp != -1)
    {
        printf("%d ", temp);
        temp = list[temp].next;
    }
    printf("\n");
}
```

```
int main()
{
    int choice, value;
```

```
initialize();

while (1)
{
    printf("\n===== MENU =====\n");
    printf("1. Insert element at beginning\n");
    printf("2. Traverse list\n");
    printf("3. Delete first element\n");
    printf("4. Display available list\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            insertAtBeginning(value);
            break;
        case 2:
            traverse();
            break;
        case 3:
            deleteFirst();
            break;
        case 4:
```

```
        displayAvailable();  
        break;  
case 5:  
    printf("Exiting...\n");  
    exit(0);  
default:  
    printf("Invalid choice! Try again.\n");  
}  
}  
return 0;  
}
```

Output :

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 1
Enter value to insert: 10
Inserted 10 at the beginning.
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 1
Enter value to insert: 20
Inserted 20 at the beginning.
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 2
Linked List: 20 10
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
```

```
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 4
Available List: 2 3 4 5 6 7 8 9
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 3
Deleted first element successfully.
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 2
Linked List: 10
```

```
===== MENU =====
1. Insert element at beginning
2. Traverse list
3. Delete first element
4. Display available list
5. Exit
Enter your choice: 5
Exiting...
```